

Title

Perform to Design Output Primitives Using OpenGL

Introduction

Computer graphics is the branch of computer science that deals with the creation and display of images using computers. Graphical objects are created using basic output primitives such as points, lines, and polygons. These primitives serve as the building blocks for designing complex figures and patterns used in applications such as animation, gaming, simulation, and computer-aided design. Understanding output primitives is essential for learning the fundamentals of computer graphics.

OpenGL (Open Graphics Library) is a widely used graphics programming library that provides functions for creating both 2D and 3D graphical objects. In this experiment, a colorful two-dimensional star design is created using polygon primitives and vertex coordinates. Different colors are applied to various sections of the star to improve its visual appearance. This experiment demonstrates the use of OpenGL functions, coordinate systems, and output primitives in constructing graphical objects and helps develop a better understanding of graphics programming concepts.

Contents

Shapes Used

- Polygon (Central diamond shape)
- Four Polygonal Stars
- 2D Geometric Shapes using vertices

Functions Used

- `glClearColor()` – Sets the background color.
- `glMatrixMode()` – Selects the projection matrix.
- `glLoadIdentity()` – Resets the current matrix.
- `glOrtho()` – Defines the orthographic viewing area.
- `glClear()` – Clears the color buffer.
- `glColor3f()` – Sets the color of the shape.
- `glBegin()` and `glEnd()` – Start and end polygon drawing.
- `glVertex3f()` – Specifies the coordinates of vertices.
- `glutSwapBuffers()` – Swaps display buffers for smooth rendering.
- `glutDisplayFunc()` – Registers the display callback function.
- `glutMainLoop()` – Starts the OpenGL event loop.

Code

```
#include <GL/glut.h>
#include <stdio.h>
#include <GL/gl.h>

void init(void)
{
    glClearColor(0.0,0.0,0.0,0.0); //GLfloat red,green,blue,alpha initial
    value 0 alpha values used by glclear to clear the color buffers
    glMatrixMode(GL_PROJECTION); // To specify which matrix is the
    current matrix & projection applies subsequent matrix to projection
    matrix stack
    glLoadIdentity();
    glOrtho(0.0, 1.0, 0.0, 1.0, -1.0, 1.0);
    //gluOrtho2D(0.0,300.0,0.0,300.0); // Orthographic representation;
    multiply the current matrix by an orthographic matrix 2D= left
    right,bottom,top equivalent near=-1,far=1
}

void Draw()
{
    /* glClear(GL_COLOR_BUFFER_BIT);
    glColor3f( 1 ,0, 0);
    glBegin(GL_POLYGON);
    glVertex
    glEnd();

    */
    // Write your Code
    glClear (GL_COLOR_BUFFER_BIT);
    /*
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_POLYGON);
    glVertex3f(0.47,0.47,0.0);
    glVertex3f(0.5,0.39,0.0);
    glVertex3f(0.53,0.47,0.0);
    glVertex3f(0.61,0.5,0.0);
    glVertex3f(0.53,0.53,0.0);
    glVertex3f(0.5,0.61,0.0);
    glVertex3f(0.47,0.53,0.0);
    glVertex3f(0.39,0.5,0.0);

    glEnd();
    */
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_POLYGON);
    glVertex3f(0.5,0.42,0.0);
    glVertex3f(0.58,0.5,0.0);
    glVertex3f(0.5,0.58,0.0);
    glVertex3f(0.42,0.5,0.0);
    glEnd();

    glColor3f(0.0, 0.1, 1.0);
    glBegin(GL_POLYGON);
    glVertex3f(0.47,0.28,0.0);
```

```

glVertex3f(0.5,0.20,0.0);
glVertex3f(0.53,0.28,0.0);
glVertex3f(0.61,0.31,0.0);
glVertex3f(0.53,0.34,0.0);
glVertex3f(0.5,0.42,0.0);
glVertex3f(0.47,0.34,0.0);
glVertex3f(0.39,0.31,0.0);

glEnd();

glColor3f(1.0, 1.0, 0.0);
glBegin(GL_POLYGON);
glVertex3f(0.47,0.66,0.0);
glVertex3f(0.5,0.58,0.0);
glVertex3f(0.53,0.66,0.0);
glVertex3f(0.61,0.69,0.0);
glVertex3f(0.53,0.72,0.0);
glVertex3f(0.5,0.8,0.0);
glVertex3f(0.47,0.72,0.0);
glVertex3f(0.39,0.69,0.0);

glEnd();

glColor3f(0.1, 1.0, 1.0);
glBegin(GL_POLYGON);
glVertex3f(0.28,0.47,0.0);
glVertex3f(0.31,0.39,0.0);
glVertex3f(0.34,0.47,0.0);
glVertex3f(0.42,0.5,0.0);
glVertex3f(0.34,0.53,0.0);
glVertex3f(0.31,0.61,0.0);
glVertex3f(0.28,0.53,0.0);
glVertex3f(0.2,0.5,0.0);

glEnd();

glColor3f(0.5, 0.5, 0.5);
glBegin(GL_POLYGON);
glVertex3f(0.66,0.47,0.0);
glVertex3f(0.69,0.39,0.0);
glVertex3f(0.72,0.47,0.0);
glVertex3f(0.8,0.5,0.0);
glVertex3f(0.72,0.53,0.0);
glVertex3f(0.69,0.61,0.0);
glVertex3f(0.66,0.53,0.0);
glVertex3f(0.58,0.5,0.0);

glEnd();

glutSwapBuffers();
}

int main(int argc,char **argv){

    glutInit(&argc,argv);
    glutInitDisplayMode ( GLUT_RGB | GLUT_DOUBLE );
    glutInitWindowPosition(0,0);

```

```
glutInitWindowSize(750,600);  
glutCreateWindow("Design Output");  
init();  
glutDisplayFunc(Draw);  
glutMainLoop();  
return 0;  
}
```

Output



Discussion

In this experiment, OpenGL output primitives were used to create a colorful 2D star design. The figure consists of a central diamond-shaped polygon surrounded by four colored star points extending upward, downward, leftward, and rightward. Different colors such as red, blue, yellow, cyan, and gray were used to improve the visual appearance of the design. The experiment demonstrates how polygon primitives and vertex coordinates can be combined to create attractive graphical patterns. The successful execution of the program verifies the effectiveness of OpenGL in designing 2D graphics and enhances the understanding of output primitives and graphical object construction in computer graphics.